

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31
- 32
- 33
- 34
- 35
- 36
- 37
- 38
- 39
- 40
- 41
- 42
- 43
- 44
- 45
- 46
- 47
- 48
- 49

How should a limited GPU allocate its expert memory and host-to-device transfer bandwidth among expert precision, residency, and lookahead, so that Mixture-of-Experts inference reduces tail latency under a quality degradation limit? Precision spends device bytes and increases the size of a later transfer, residency spends device bytes to avoid a transfer, and lookahead spends both bytes and bandwidth to hide a transfer that still occurs. Existing policies usually fix two of these uses and optimize the third, so a precision quota or a prefetch depth tuned on its own can still raise tail latency once the two budgets are shared. DynaByte answers this question with donor-backed byte exchanges: each request names the precision, residency, or lookahead claim it displaces, and a deadline replay admits the exchange only when predicted exposed wait falls and the quality-degradation limit still holds. One byte ledger and a publish-after-copy protocol keep the expert-memory peak within budget while the three uses are admitted together.

Additional Key Words and Phrases: mixture of experts, memory management, prefetching, quantization, performance modeling

Anonymous Author(s). 2027. DynaByte: Allocating Expert Memory and Transfer Bandwidth for MoE Tail Latency. *Proc. ACM Meas. Anal. Comput. Syst.* 1, 1 (October 2027), 25 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Draft status. This manuscript follows the SIGMETRICS 2027 acmsmall rules (at most 20 single-column pages of technical content, unlimited references, double-anonymous review). Tables that depend on the joint re-measurement are marked in red and left blank. The motivating measurements and trace replay in Sections 2.2–2.5 are not end-to-end results for DynaByte.

A Mixture-of-Experts (MoE) layer routes each token to a small subset of expert subnetworks, allowing model capacity to grow without a proportional increase in per-token computation [10, 19, 24]. The memory footprint remains large because every expert representation must still be stored or made available at inference time. Models such as Mixtral, DeepSeek-V2, and Qwen3 contain tens to hundreds of billions of parameters while activating only a fraction of them for each token [6, 15, 29]. On a single accelerator, expert weights compete with non-expert parameters, the KV cache, activations, and runtime workspaces for device memory. When experts cannot all remain at the desired precision, the system must both compress them and move them across the host-to-device link.

How should a limited GPU allocate its expert memory and host-to-device transfer bandwidth among expert precision, residency, and lookahead, so that Mixture-of-Experts inference reduces tail latency under a quality-degradation limit?

These two budgets meet in three uses. Precision spends expert-memory bytes on a higher-precision representation and increases the size of any later transfer. Residency spends bytes to keep an expert on the device so that a later access requires no transfer. Lookahead spends both bytes and bandwidth to stage an absent expert early enough that an otherwise unavoidable transfer overlaps

preceding computation. A byte assigned to precision cannot also retain another expert or stage a future one. Host-to-device bandwidth determines whether that early transfer finishes before its deadline.

Prior work has developed effective policies for parts of this problem. Quantization and mixed-precision methods choose expert representations under a memory or accuracy target [8, 11, 21]. Expert caches and prefetchers decide which experts to keep and when to transfer missing experts, commonly under a fixed representation size [9, 25, 27, 32]. Mixed-precision offloading systems demonstrate that lower-precision transfers, caching, and prefetching can coexist [13, 28]. The right assignment still changes with the workload: expert sensitivity sets the value of precision, routing locality sets the value of residency, and transfer deadlines set the value of lookahead. A fixed partition or a fixed priority among the three uses cannot express these changing opportunity costs. These systems show that the three mechanisms can operate together, yet they do not allocate expert memory and transfer bandwidth jointly among precision, residency, and lookahead under one quality-degradation limit.

DynaByte answers this question. Under a hard expert-memory budget B and a quality-degradation limit ϵ , it selects expert precision, residency, and prefetch timing to reduce tail latency. The decision unit is a byte exchange: a requested expert state together with the precision, residency, or lookahead claim that would fund it. Its value is the change in predicted exposed wait per requested byte after applying the complete exchange. Quality risk is a feasibility constraint on that resulting state, rather than a latency reward. DynaByte represents each memory claim as a time-bounded *byte lease* that records its owner, size, purpose, and expected lifetime. A state monitor tracks routing demand, reuse, transfer deadlines, and quality-risk estimates. It also measures *in-overlap bandwidth*, the copy rate observed while expert computation is active. A joint coordinator pairs requested leases with explicit *donor leases*, the existing claims released to fund a request, and replays the deadline-ordered transfer queue before and after each exchange. It admits only exchanges that improve predicted exposed wait while satisfying the quality-risk and peak-memory accounts. Residency and lookahead refresh on a short time scale, and precision on a longer window, so that a precision change can amortize its migration cost. Both rates use one admission path and the same byte ledger, which records published and reserved expert bytes. Neither rate receives a private quota. The executor reserves memory before a copy begins, publishes a new expert representation only after the copy completes, and reclaims the old representation after its readers finish.

We make three contributions.

- To answer this question, we formulate the joint allocation of expert memory and host-to-device transfer bandwidth among precision, residency, and lookahead. The objective is tail latency under a quality-degradation limit and a peak expert-memory budget. The formulation exposes opportunity costs hidden by independent precision, caching, and prefetch policies.
- We develop a byte-lease exchange policy that names the memory displaced by each request and counterfactually replays the resulting deadline schedule. Joint replay prevents residency and lookahead from receiving credit for the same avoided transfer and charges precision choices for their transfer cost.
- We design a runtime with one byte ledger and a publish-after-copy protocol. It admits transitions only when their peak footprint fits and never exposes an incomplete representation to the forward path.

2 BACKGROUND AND MOTIVATION

2.1 Expert Memory on One Accelerator

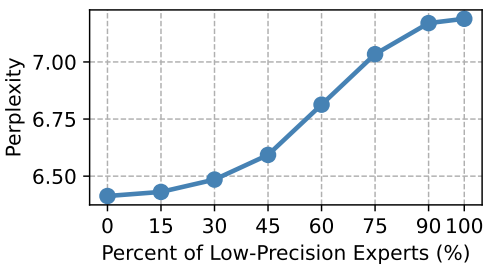
An MoE layer ℓ contains E_ℓ routed experts and sends each token to k_ℓ of them. The router determines which experts execute, but it does not determine where their weights are stored or when they arrive on the device. If a selected expert is absent, its weights must be transferred from host memory before its computation can finish. A demand transfer can expose most of this cost to the forward pass. A prefetch can hide part of the transfer by issuing it before the expert is needed.

We reserve memory for non-expert parameters, the KV cache, activations, and runtime workspaces before assigning memory to experts. The remaining capacity is the expert-memory budget B . Host memory stores packed low- and high-precision representations of every routed expert. Their measured sizes are $m_{\ell,e}^{\text{lo}}$ and $m_{\ell,e}^{\text{hi}}$; nominal bit width is not used for accounting. Let $P_b = \sum_{\ell,e} m_{\ell,e}^b$ be the aggregate footprint at tier $b \in \{\text{lo}, \text{hi}\}$. We use $\Phi = B/P_{\text{lo}}$ for the budget relative to the low-tier footprint and $\rho = P_{\text{hi}}/P_{\text{lo}}$ for the high-to-low footprint ratio. At any instant, an expert is absent, resident at low precision, resident at high precision, or in transition to a new published state. The forward pass can use only a published resident representation.

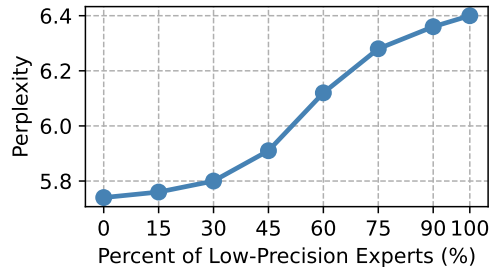
The host-to-device bandwidth budget is the measured copy rate on the migration path. It is the rate of one queue, not a second capacity that the runtime partitions. An exchange allocates that rate by choosing a representation size, whether the expert stays resident, and when its transfer occupies the queue. Allocating expert-memory bytes therefore decides which transfers consume the measured bandwidth.

Those two budgets meet in three uses. Precision assigns extra bytes to a higher-precision representation. We call that increment *fidelity* and use fidelity for this use in the rest of the paper. *Residency* keeps an expert on the device so that a later access requires no transfer. *Lookahead* stages an absent expert early enough to overlap its transfer with preceding computation. Lookahead is not a separate representation: it is a timed transition from absence to residency. Residency avoids a transfer. Lookahead reduces the critical-path portion of a transfer that still occurs.

The four measurements below are one argument about this allocation. The first prices a fidelity quota and records that the higher-precision copy is the larger transfer. The second shows that the experts a workload uses change across tasks, so neither the quota's recipients nor the resident set can be frozen from one task. The third holds representation size fixed and shows when host-to-device bandwidth can hide the experts that remain absent. The fourth places all three uses on one expert-memory budget, at one measured bandwidth, and shows that moving fidelity moves residency and lookahead together.



(a) Qwen3-30B: FP16/INT4



(b) Qwen3-Next-80B: INT4/INT2

Fig. 1. WikiText-2 perplexity under frozen precision-quota sweeps. Experts are demoted in the order defined by an independent, fixed ranking; these are not online-policy results.

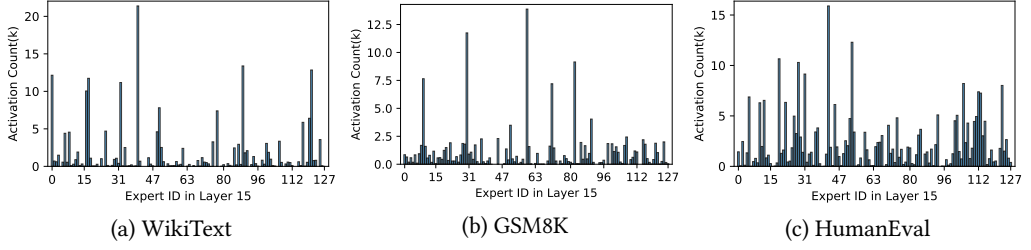


Fig. 2. Selected token-expert dispatch counts at Layer 15 of Qwen3-30B. All experts remain at low precision and the counters are reset between workloads.

2.2 Fidelity Has a Quality Price and a Transfer Cost

Releasing expert-memory bytes by lowering precision also shrinks later transfers, but the released bytes have a quality price. Figure 1 varies the fraction of low-precision experts under a frozen expert ranking.

Each point demotes a suffix of the same ranking, so the curves isolate the fidelity quota rather than online reassignment. Perplexity rises continuously on both models. Raising the fidelity share therefore spends expert memory and, because the high-precision representation is the larger packed size, increases the bytes a later transfer must move. That continuous rise is why the share sits under a quality-degradation limit. The sweep uses one frozen ranking and does not name the experts who receive the higher-precision bytes. The next measurement asks whether the experts a workload uses stay fixed.

2.3 The Experts a Workload Uses Also Change

The quality curve prices a fidelity quota without naming its recipients. Figure 2 shows that the experts actually dispatched at Layer 15 change across text, math, and code. Every expert stays at low precision, and the counters reset between workloads. A fidelity assignment or a resident set taken from one workload therefore misses experts the next workload uses. The counts measure access frequency, not quantization sensitivity, so they do not rank the quality harm of a low-precision expert. They do make both the fidelity recipients and the resident set workload-dependent inputs to the same allocation.

2.4 Whether the Remaining Bytes Can Be Hidden Also Changes

The quality curve prices the fidelity quota, and the dispatch counts show that the accessed experts change with the task. What remains open is whether an expert that is still absent can arrive before it is used. High expert demand does not by itself imply that lookahead can hide a transfer. Figure 3 compares the bytes selected in one MoE layer with an aggregate upper bound on bytes transferable during that layer's compute window. Demand comes from five measured prompt blocks at each batch size. The bound multiplies measured blocking host-to-device (H2D) bandwidth by mean TTFT or TPOT divided by 48 routed layers. It is deliberately optimistic: it ignores individual expert deadlines and bandwidth lost to concurrent kernels. Prefill touches most routed experts, yet its compute window grows with the batch. Decode touches fewer experts, but its shorter window leaves less time before use. Demand determines what may need to move; the compute window and the expert's deadline determine whether moving it early is useful. Lookahead's bandwidth cost is the gap between selected bytes and the bytes this host-to-device rate can move inside the window. Prefill and decode move that gap in opposite directions, so one lookahead depth does not

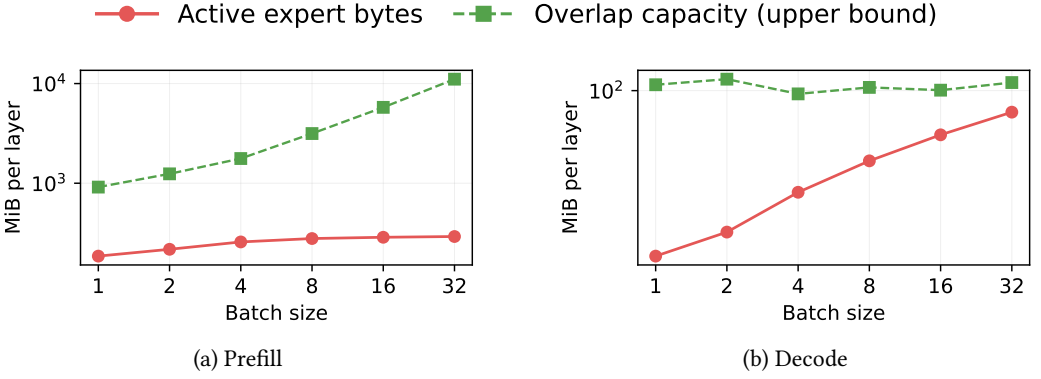


Fig. 3. Active expert bytes and an upper bound on overlap capacity for Qwen3-30B. The bound uses measured blocking-copy bandwidth and mean per-layer TTFT or TPOT; it is not the result of an overlapping offload runtime.

fit both phases. This comparison holds representation size fixed. The next measurement lets fidelity, residency, and lookahead draw from one budget.

2.5 A Fidelity Change Moves Residency and Lookahead

None of the measurements above lets the three uses compete for one budget. Figure 4 places fidelity, residency, and lookahead on the same expert-memory budget, at the measured transfer bandwidth of the replay. The experiment replays a measured 2,048-token routing trace under the byte-accurate expert budget used by our Qwen3-30B runtime. It imposes an illustrative WikiText-2 limit of 2.1 perplexity above the all-high configuration. Among the scanned high-precision fractions, only 25% and 40% satisfy both the quality-degradation limit and the expert-memory budget. Residents follow the frequency ranking from warm-up traces; lookahead has oracle knowledge of the measured active set but is limited by its staging slots and the measured transfer window.

At 25% high precision, 12 staging slots leave 96 low-precision residents per layer and reduce exposed transfer to 0.44 ms/request. Raising the high-precision share to 40% leaves only 43 low-precision residents at its best staging point. Exposed transfer is minimized at eight slots and then rises because additional staging displaces residents. Changing fidelity therefore changes both residency and the useful amount of lookahead. Host-to-device bandwidth is fixed in this replay. The figure shows the memory coupling at one measured rate; it does not sweep both budgets.

2.6 Motivation and Design Challenges

Together, the four measurements are one allocation problem. Fidelity has a quality price and enlarges later transfers. The experts a workload uses change with the task, so the fidelity recipients and the resident set change with it. Whether an absent expert's transfer can be hidden changes with host-to-device bandwidth and the compute window. Moving the fidelity share, at a fixed bandwidth, moves both residency and the lookahead depth that reduces exposed transfer. Existing mechanisms may provide quantization, caching, and prefetching in the same system without deciding which use has the greatest opportunity cost in the resulting expert state. A suitable policy assigns expert memory and transfer bandwidth among fidelity, residency, and lookahead by marginal value, under one quality-degradation limit, to reduce tail latency.

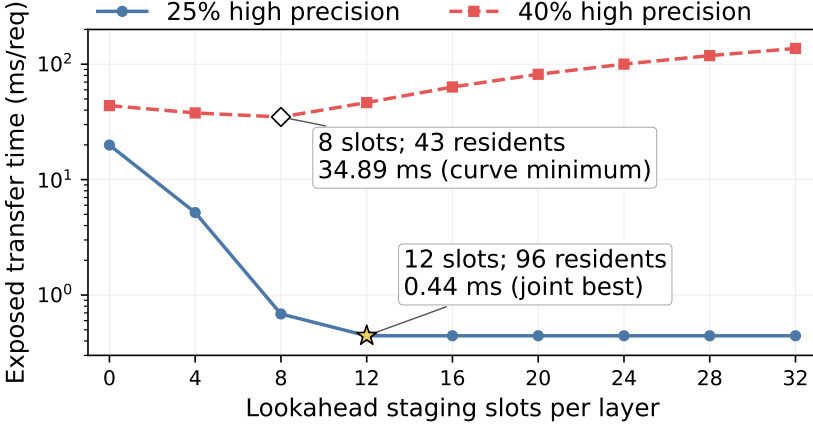


Fig. 4. Trace replay of the two quality- and memory-feasible precision allocations on Qwen3-30B, at the measured host-to-device bandwidth of this trace. Increasing the high-precision share from 25% to 40% reduces residency and moves the best staging allocation from 12 to 8 slots. Bandwidth is not swept. Lookahead uses oracle knowledge of the measured active set; this is not an end-to-end DynaByte result.

Challenge 1: heterogeneous value under a quality-degradation limit. Fidelity reduces quality risk, residency avoids a future transfer, and lookahead reduces the exposed portion of a transfer that still occurs. Their benefits have different units and durations. An action comparison must account for byte cost and transition cost without treating task quality as an unconstrained latency currency.

Challenge 2: future-dependent value at different time scales. Fidelity depends on quantization sensitivity and the probability that an expert executes. Residency depends on reuse, while lookahead depends on an expert's deadline, the available compute window, and effective transfer bandwidth. Routing locality and deadlines can change faster than the evidence needed to justify a fidelity transition. Both budgets are still decided in the same exchange. Fidelity requests refresh more slowly only because their quality evidence and copy cost take longer to amortize.

Challenge 3: hard memory safety during transitions. A feasible final resident set does not guarantee a feasible transition. Promotion and prefetching require destination bytes before a copy begins, and an old published representation may remain live until its readers finish. An in-flight representation cannot serve the forward pass. The runtime must therefore account for published and reserved bytes together and publish a replacement only after its transfer completes.

A runtime must therefore represent all three uses in one byte account, compare each requested state with the states displaced to fund it, and commit the exchange only when the expert-memory budget holds and the replayed migration queue prices the bandwidth that exchange consumes.

3 DESIGN

3.1 Overview

DynaByte answers the allocation question with one admission procedure. It represents a change of precision, residency, or lookahead as a donor-backed byte exchange, scores that exchange by the predicted exposed wait it saves per byte, and admits it only while the quality-risk account and

the expert-memory ledger both hold. One executor then commits the admitted schedule. Host-to-device bandwidth has no separate quota: the deadline replay charges each exchange for the migration-queue time its transfers occupy.

The procedure has five steps.

- (1) Observe one snapshot of routing demand, quality risk, deadlines, bandwidth, and byte state.
- (2) Generate candidate leases for fidelity, residency, and lookahead inside one candidate envelope.
- (3) Attach donor leases to each request and score the complete exchange with one deadline replay.
- (4) Admit exchanges in decreasing saved wait per byte while both accounts remain feasible.
- (5) Commit each admitted exchange by reserving, copying, publishing, and reclaiming on one arena.

The four components in the rest of this section implement these steps. None of them optimizes tail latency on its own.

Every precision, retention, and prefetch decision is the exchange

$$x = (Z^+, D, \mathcal{T}), \quad (1)$$

where Z^+ is the compatible set of leases to create or renew, D is the set of leases released to fund them, and $\mathcal{T}$ is the transition schedule during which old and new representations may coexist. The coordinator evaluates the state after the whole exchange, so the value of one use includes its effect on the other two. Quality risk and peak bytes are feasibility checks on that state. A shared ledger reserves the complete schedule $\mathcal{T}$, so concurrent exchanges cannot spend the same released bytes.

Optimization contract. Let a runtime policy π choose the representation, resident set, and transfer issue time of every expert. Expert memory enters as the hard budget B . Host-to-device bandwidth is the measured rate of the migration queue replayed below, the same rate defined in Section 2.1. The objective constrains expert memory and quality; bandwidth is accounted for inside that replayed schedule.

$$\min_{\pi} \text{P95}(\text{TPOT}; \pi) \quad \text{subject to} \quad M_{\text{peak}}(\pi) \leq B, \quad \Delta Q(\pi) \leq \epsilon. \quad (2)$$

Background reserves non-expert weights, the KV cache, activations, and fixed workspaces before deriving B . The quality-degradation limit is measured against a fixed reference configuration for each task. The runtime uses a calibrated risk estimate to guide decisions; Section 6.2 states why that estimate is not a per-request quality guarantee.

An expert $i = (\ell, e)$ is absent, resident at low precision, resident at high precision, or in transition toward one of the resident states. Let its packed footprints be m_i^{lo} and m_i^{hi} . A high-precision resident consumes a low-precision residency footprint plus the fidelity increment

$$\delta m_i^{\text{fid}} = m_i^{\text{hi}} - m_i^{\text{lo}}. \quad (3)$$

The decomposition is logical; a high-precision representation remains one packed block. Only a published resident representation is executable. A destination under transfer is reserved but not published, and an old representation with outstanding readers remains published until reclamation.

One exchange, end to end. Consider an absent expert predicted two layers ahead when no free block remains. The snapshot records the expert as absent, names its deadline, and reports that the byte ledger has no slack. The generator emits a lookahead lease for the selected representation. The coordinator pairs that request with the resident it would evict and, when more bytes are required, with the fidelity increment it would release. One replay compares predicted exposed wait before and after the exchange. Admission requires the saved wait to exceed the donor's future miss and

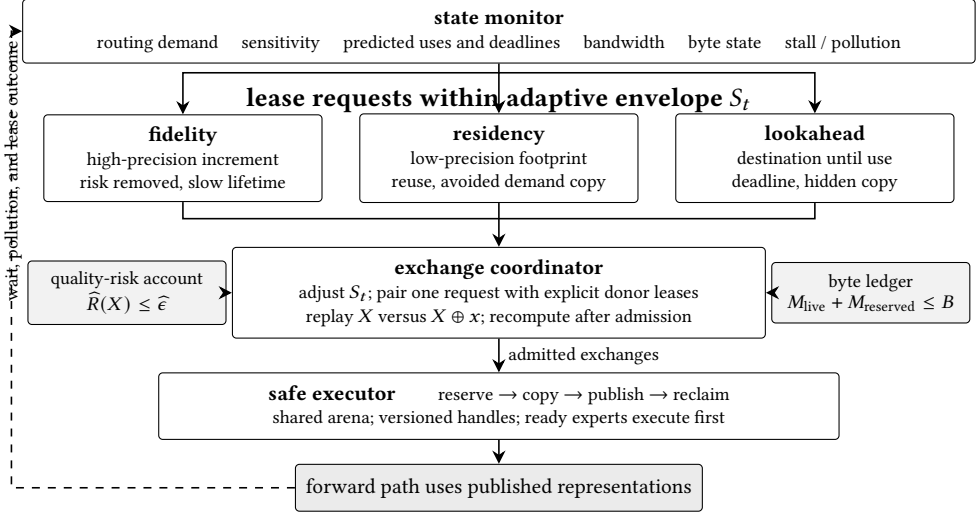


Fig. 5. DynaByte runs one admission procedure. The state monitor builds the snapshot (step 1). Fidelity, residency, and lookahead leases are inputs to one exchange coordinator, which scores each complete exchange and admits it (steps 2–4). The safe executor commits the admitted schedule (step 5).

Table 1. Lease transitions in the common action space. “Release” occurs only after the transition in the last column makes the bytes reclaimable.

Current	Requested	Lease change	Transition
absent	in flight (low/high)	acquire lookahead of m_i^{lo} or m_i^{hi}	reserve, copy
resident (low)	resident (high)	acquire fidelity of δm_i^{fid}	promote, publish
resident (high)	resident (low)	release fidelity	demote, publish
resident (low/high)	absent	release residency and any fidelity	drain readers, reclaim
lookahead	resident (same tier)	convert lookahead into residency and any fidelity	publish, reclassify

migration cost, the provisional quality-risk account to hold, and the peak footprint of the transition to fit B . The executor then reserves the destination, copies into it, publishes the new representation, and reclaims the donor after its readers finish. The same five steps apply when the request is a fidelity upgrade or a residency renewal; only the donor and the term changed in the replay differ.

Figure 5 assigns the steps to four components. The state monitor builds the snapshot in step 1. The lease generator and valuator produce the candidate leases in step 2 and compute the score used in step 3. The exchange coordinator attaches donors and performs the admission in step 4. The safe executor commits the schedule in step 5.

Table 1 is the action space of step 2. Low- and high-precision loads of an absent expert are mutually exclusive candidates. A high-precision load occupies one physical block, although its resident state is accounted logically as a base residency lease plus a fidelity increment.

3.2 State Monitor

The state monitor's only output is the snapshot consumed by step 1. It selects no exchange and promotes no expert.

For each expert, it distinguishes the published state used by dispatch, a reserved transition accepted by the executor, and the provisional state considered by the coordinator. It also records recent uses, predicted next-use time, and the device location of every published or reserved representation. Per-layer counters record expert-kernel time, in-overlap bandwidth, and exposed wait attributed to missing experts. The monitor reports published and reserved bytes separately. A provisional state consumes no capacity until admission. A destination bound by a reserved transition does.

Two measurements tell the coordinator whether the candidate envelope should grow or shrink. The stall signal s_t is the fraction of routed-layer time exposed while waiting for expert transfers during fast period t . We define *lookahead pollution* p_t as the sum of unused lookahead bytes and bytes reloaded after a speculative displacement, divided by $\max(B_t^{\text{look}}, b_{\min})$. Here B_t^{look} is the lookahead traffic admitted in the period and $b_{\min}$ is the smallest packed expert block. The monitor smooths both signals:

$$\tilde{s}_t = \beta \tilde{s}_{t-1} + (1 - \beta) s_t, \quad \tilde{p}_t = \beta \tilde{p}_{t-1} + (1 - \beta) p_t. \quad (4)$$

Stall indicates that the envelope may be too short; pollution indicates that looking farther ahead is consuming bytes without hiding useful work. Keeping the two measurements separate distinguishes a late correct prediction from an early false positive. Section 3.4 applies them; the monitor does not update the envelope.

The quality-risk estimate separates the harm of an expert's low representation from the frequency with which that representation is expected to execute. An offline sensitivity $q_i \geq 0$ measures the nonnegative increase in calibration loss when only expert i uses the low representation. The runtime does not modify q_i . After a forward step with N_t routed positions, it measures expert e 's routing demand as

$$c_{t,e} = \frac{1}{N_t} \sum_{t=1}^{N_t} \mathbf{1}[e \in \mathcal{R}_{t,t}] p_{t,t,e}, \quad (5)$$

where $p_{t,t,e}$ is the router weight. The monitor applies $w_i \leftarrow \alpha w_i + (1 - \alpha) c_i$. The quantity w_i is a routing weight, not a quality score.

For a planned state X , let $h_i(X) = 1$ when expert i 's next execution uses the high representation and zero otherwise. The monitor computes

$$\widehat{R}(X) = \sum_i w_i q_i (1 - h_i(X)). \quad (6)$$

A proxy limit $\widehat{\epsilon}$ is chosen on the calibration split together with the actual quality-degradation limit ϵ . The coordinator uses $\widehat{R}(X) \leq \widehat{\epsilon}$ as a feasibility check and never converts risk into milliseconds.

The same snapshot stores recent routing transitions and, when available, pre-gating (pregate) logits emitted before the target MoE layer. The generator in Section 3.3 turns those logits into use probabilities. The snapshot also stores predicted expert uses, their earliest issue times and deadlines, reuse probabilities, and the measured bandwidth available to the migration stream. Completed, late, unused, and eviction-induced reloads update these estimates. Feedback changes later valuations but cannot alter the validity of the published state. The resulting snapshot is the only interface to the following components.

3.3 Lease Generator and Valuator

The generator and valuator perform step 2 and compute the score used in step 3. They admit nothing.

A lease $z = (u, i, b, [t_s, t_f])$ records its use $u \in \{\text{fid}, \text{res}, \text{look}\}$, expert i , byte count b , and expected lifetime. The lifetime is the interval over which the valuator estimates benefit. It guides valuation rather than guaranteeing that occupancy ends at exactly t_f . An early use can shorten a lookahead lease, and a late transfer retains its bytes until completion or cancellation.

The valuator scores a complete exchange with one deadline-ordered replay. For predicted use j , the snapshot supplies expert i_j , representation tier b_j , size $m_{i_j}^{b_j}$, and occurrence probability ω_j . Its *deadline* d_j is the latest ready time that introduces no transfer wait for that use. A resident expert is ready at the start of the window. For an absent expert, completion time $r_j(X)$ includes bytes already queued on the migration stream and the earliest time at which its copy can issue. Transfers are replayed in earliest-deadline-first order. Under continuous batching, uses of the same expert and tier share one readiness event: the replay schedules its transfer once, then evaluates every request-specific deadline against that completion time. Retention can therefore remove one shared transfer without receiving duplicate credit for each copy that would never have occurred. The resulting exposed wait is

$$\widehat{L}(X) = \sum_j \omega_j [r_j(X) - d_j]_+. \quad (7)$$

Calibration relates this quantity to measured tail latency. It is a scheduling surrogate for Eq. (2), not a direct estimate of the P95 quantile.

After the coordinator attaches donor leases, the valuator receives the complete exchange x . Its net value is

$$G(x \mid X) = \widehat{L}(X) - \widehat{L}(X \oplus x) - C_{\text{move}}(x), \quad (8)$$

where $X \oplus x$ applies the request, donor releases, and schedule in Eq. (1), and C_{move} is the exposed transition cost. The same G scores every use. Fidelity changes the transfer size inside the replay, residency removes a transfer, and lookahead changes the issue time. Joint replay gives no lookahead credit to an expert already retained and charges a high-precision transfer for its larger representation.

Table 2 records the bytes and lifetime that each use contributes to this replay. A fidelity lease claims δm_i^{fid} bytes beyond the residency lease and normally spans a slow control period, so its quality benefit can amortize the transition. A demotion offer releases the increment only after the low-precision replacement is published. A residency lease claims m_i^{lo} base bytes for one fast control period, independently of any fidelity increment. Renewal is credited when predicted reuse removes a demand transfer that lookahead would not already hide. An eviction offer ends the lease after outstanding readers finish. For an absent expert, the generator emits mutually exclusive low- and high-precision lookahead alternatives when both satisfy the quality-risk account. Each claims the full footprint of its representation from the earliest issue time through predicted use, and its value is only the part of the transfer removed from the critical path. After use, an accepted lookahead lease either converts in place to residency, adding a fidelity increment for a high representation, or expires after dispatch and reclamation.

The generator limits those candidates to the adaptive envelope S_ℓ , the maximum routed-layer depth allowed to generate them. It initializes the envelope from the amount of work that can overlap one layer:

$$S_0 = \text{clip}_{[S_{\min}, S_{\max}]} \left(\left\lceil \frac{N_e E_s}{CT_\ell} \right\rceil \right), \quad (9)$$

Table 2. Fields of one lease. The replay effect is the term of Eq. (8) that the use changes.

Use	Bytes	Lifetime	Replay effect
Fidelity	δm_i^{fid}	slow period	transfer size
Residency	m_i^{lo}	one fast period	whether the transfer exists
Lookahead	m_i^{lo} or m_i^{hi}	issue until use	issue time

where N_e is the mean number of predicted absent experts per layer and E_s is their mean selected representation size. The estimate initializes S_t . Section 3.4 then adjusts it from stall and pollution. The envelope bounds candidate generation only. It reserves no memory and sets aside no lookahead quota.

For each layer inside the envelope, a CPU-side forecaster reads the pregate logits stored in the snapshot when the checkpoint exposes them, and otherwise uses recent routing-transition frequencies. It converts the base scores into a use probability ω_j and a confidence score calibrated on warm-up traces. Low confidence admits a wider set of possible experts without extending the envelope. Predictions are cached by request prefix, layer, and envelope, and a late forecaster falls back to the base routing signal so it never delays the forward path. An optional CPU correction model can learn the deviation between the base prediction and realized activation. The admission rule does not use it; Section 5 evaluates it separately.

An aggregate bound prunes prefixes inside S_t that cannot cover their predicted miss bytes. For resident set R and predicted expert set $\hat{U}(S)$ over the next S routed layers,

$$M_{\text{miss}}(S) = \sum_{i \in \hat{U}(S) \setminus R} m_i. \quad (10)$$

With measured per-layer compute time T_t and in-overlap bandwidth C , the generator retains only prefix depths satisfying

$$M_{\text{miss}}(S) \leq ST_t C. \quad (11)$$

The generator searches only $S \leq S_t$. Equation (11) prunes candidate prefixes before scoring. Equation (8) still uses individual issue times and deadlines, and the coordinator decides how many lookahead bytes to admit.

3.4 Exchange Coordinator

The exchange coordinator attaches the donors in step 3 and admits exchanges in step 4. It consumes the snapshot and the lease requests, maintains a provisional lease state X , and submits admitted exchanges to the executor.

Two accounts constrain X . At every step the byte account requires

$$M_{\text{live}}(t) + M_{\text{reserved}}(t) \leq B \quad \forall t, \quad (12)$$

and currently uncommitted capacity is

$$B_{\text{slack}} = B - M_{\text{live}} - M_{\text{reserved}}. \quad (13)$$

The executor maintains the authoritative ledger; the coordinator uses the same entries for provisional admission. The quality-risk account requires $\widehat{R}(X) \leq \widehat{c}$. One state and one ledger give two consequences. A representation has one readiness event in the replay, so residency and lookahead cannot both claim the same avoided transfer. A donor release is credited at its scheduled reclaim time, so no two exchanges can spend the same bytes or spend future bytes as current slack.

Any of the three lease types can fund any request. For a request that does not fit in B_{slack} , the coordinator constructs D from fidelity leases eligible for demotion, residency leases eligible for eviction, and lookahead leases eligible for cancellation or expiry. It first considers reclaimable leases: lookahead copies that have not started or are no longer useful, and residents whose predicted reuse lies beyond the current envelope. A recently used resident or one predicted inside the envelope is protected. Such a lease can still become a donor when the complete replay includes its later reload and the exchange remains positive after paying that cost. A complete exchange must satisfy four conditions: its request and donors do not conflict on an expert; its resulting risk stays below $\hat{\epsilon}$; its transition schedule satisfies Eq. (12); and its transfers respect their release times and deadline order. Among feasible donor sets, the coordinator keeps the one with the smallest increase in predicted wait and migration cost. For each request, that search scans leases in increasing *counterfactual loss*, the increase in predicted exposed wait and migration cost caused by releasing the lease, and stops after finding enough releasable bytes. Bytes that become reclaimable later appear at their release time and are never counted as current slack.

Let b_x^{req} be the bytes claimed by exchange x 's requested lease for its next lifetime, including bytes renewed by a residency request. The coordinator orders feasible exchanges with positive G by

$$\eta(x \mid X) = \frac{G(x \mid X)}{b_x^{\text{req}}}. \quad (14)$$

After each admission, it updates the provisional byte and risk accounts and asks the valuator to recompute affected exchanges. Revaluation is necessary because retaining an expert can eliminate its lookahead request, while a precision change can alter several completion times. The candidate set contains only resident experts, experts predicted inside the current envelope S_t , and experts eligible for a precision change. Mutually exclusive tier alternatives for one expert conflict, as do exchanges that release or renew the same lease. The resulting bounded, receding-horizon search is an online approximation of Eq. (2).

A workload shift can leave the published state above $\hat{\epsilon}$. Before ranking latency exchanges, the coordinator projects the state back into the quality-risk account by promotion exchanges with the smallest predicted latency cost per unit of risk removed. The projection uses the same replay as G . It is the feasibility step of this admission path, not a separate precision objective. Once the account is feasible, a demotion can fund another request only if the complete exchange remains feasible. The repair can consolidate fidelity bytes: promoting one sensitive, frequently used expert may allow several lower-risk fidelity leases to expire and release net capacity.

The same path refreshes candidates at two rates. Every T_{fast} routed layers, it refreshes deadline, reuse, bandwidth, residency, and lookahead state. Every T_{slow} forward steps, it also refreshes routing weights and adds fidelity requests to that same pool. A fidelity lease accepted at a slow event remains visible to every fast event, and repeated fast misses contribute to its opportunity cost at the next slow event. A minimum fidelity-lease tenure and replacement hysteresis suppress changes whose predicted gain is too small to repay their copies.

The coordinator also maintains the candidate envelope that limits step 2. At each fast event it applies

$$S_{t+1} = \begin{cases} \min(S_t + 1, S_{\max}), & \tilde{s}_t - \lambda \tilde{p}_t > \theta_h, \\ \max(S_t - 1, S_{\min}), & \tilde{s}_t - \lambda \tilde{p}_t < -\theta_h, \\ S_t, & \text{otherwise.} \end{cases} \quad (15)$$

The one-layer step and hysteresis band keep short bursts from changing the search depth repeatedly. The update decides which future experts become candidates. Byte exchange and deadline replay still decide admission.

Algorithm 1 is this path in execution order. It applies Eq. (15) before generating candidates. On a slow boundary it adds fidelity requests and applies the quality-risk projection above. The loop then scores fidelity, residency, and lookahead exchanges with the same η .

Algorithm 1 One DynaByte control event.

Require: monitor snapshot, published and reserved leases, byte ledger, envelope S_t

- 1: read stall and pollution exponentially weighted moving averages
 - 2: update S_t using Eq. (15)
 - 3: forecast uses within S_t ; refresh the deadline-ordered transfer replay
 - 4: **if** this event reaches a slow boundary **then**
 - 5: update routing weights using Eq. (5)
 - 6: generate eligible fidelity requests and donor leases
 - 7: project X into $\widehat{R}(X) \leq \widehat{\epsilon}$ by minimum-cost promotions
 - 8: **end if**
 - 9: generate residency and lookahead requests within S_t
 - 10: **for all** eligible fidelity, residency, and lookahead requests **do**
 - 11: attach the least-cost donor set, preferring reclaimable leases
 - 12: discard exchanges that violate risk, bytes, deadlines, or expert state
 - 13: **end for**
 - 14: **while** a feasible exchange has positive $G(x | X)$ **do**
 - 15: choose the exchange with maximum $\eta(x | X)$
 - 16: add it to the provisional state and update both accounts
 - 17: recompute affected completion times and exchange values
 - 18: **end while**
 - 19: submit admitted exchanges for atomic reservation and execution
-

3.5 Safe Executor

The safe executor performs step 5. It receives an exchange the coordinator has already admitted and turns that exchange into published expert state while enforcing Eq. (12). It does not choose among fidelity, residency, and lookahead.

Device expert memory is one shared arena managed by a coalescing extent allocator. An *arena extent* is a contiguous allocation bound to one packed representation. Its size is the measured byte span rounded to the allocator granularity. Free lists index compatible extents but do not own capacity by layer, representation tier, or lease purpose. Fidelity, residency, and lookahead can therefore exchange byte credits across size classes rather than compete only inside fixed high- and low-precision quotas. An in-flight destination is an ordinary reserved arena extent, so its bytes remain inside B .

The executor keeps protected and reclaimable indexes over these same extents and reports both to the coordinator, which remains responsible for choosing donors. A hit or a predicted use inside S_t places a resident in the protected index. An expired or unused lookahead lease and a resident whose reuse lies beyond the envelope enter the reclaimable index. Recency orders extents only within each index, so LRU breaks ties inside an index.

Before issuing a copy, the executor reserves the exchange's complete transition schedule in one ledger transaction protected by the ledger lock. The schedule records every interval in which a source and its replacement coexist, together with the publication and reclamation events that end that interval. A destination that must exist before a donor can be reclaimed is bound to an actual arena extent during reservation. An exchange is refused if no compatible extent can be

bound even when the aggregate byte count fits. A demotion cannot fund another destination until its low-precision replacement is published and the high-precision block becomes reclaimable; an eviction-only donor may be reclaimed first after its readers finish. If the schedule cannot reserve its peak footprint, the executor cancels the exchange and leaves the published state unchanged. The provisional state is discarded, and the next control event replans from the current published and reserved snapshot. A bounded submission queue applies backpressure before reservation, so concurrent exchanges cannot spend the same future bytes.

Publication. A *published handle* contains a monotonically increasing version and an immutable representation descriptor. The descriptor holds the tier tag, device pointer, and quantization metadata. The forward path acquires a reader reference before launching an expert kernel, records the latest last-use event for that compute stream, and then releases the reference. After the copy event completes, the executor replaces the registry entry while holding the registry lock. It reclaims the old extent after its reader count reaches zero and every recorded last-use event completes, covering the interval between reading a pointer and launching its kernel. Only the latest event per compute stream is retained, so this metadata is bounded by serving concurrency. A precision transition continues to serve from the old representation and does not block the forward path.

Resident-first execution. Routing semantics and the selected top- k experts do not change. After routing, the executor groups tokens by selected expert and issues groups whose representations are already published before groups waiting on transfers. If a lookahead transfer misses its deadline, only the assigned group waits for the remaining copy while other ready groups proceed. This order uses the published state; it does not choose which expert occupies memory.

A speculative lookahead exchange cannot evict a protected resident unless its replay explicitly schedules and charges the resident's later reload. Within the current envelope, a false positive therefore consumes its own copy and lease but cannot silently trigger an unpriced chain of evictions and reloads. The executor returns completion time, late bytes, unused lookahead bytes, eviction-induced reload bytes, and exposed wait to the monitor for the next control event.

4 IMPLEMENTATION

We implement DynaByte in Python using PyTorch and HuggingFace Transformers [23, 30]. The CPU decision plane evaluates byte exchanges, while the device-memory path owns representation storage, publication, and reclamation.

Controller. A predicted use records its expert, representation tier, issue layer, and use layer. The implemented predictor ranks experts from recent layer-local activation counts and limits its output to a configured width. Replay uses measured layer time and copy bandwidth to simulate one migration queue and compute readiness and exposed wait from CPU-side placement metadata. At each routed layer, one admission path forms lookahead candidates and updates the adaptive candidate envelope from observed stall and pollution. A slower interval on that same path refreshes fidelity requests. Fidelity tenure and replacement hysteresis are stored with their byte leases.

Representations. Both representation tiers are prepared in pinned host memory before a timed run. Packing proceeds one layer at a time and releases that layer's source weights after both tiers are complete. The tier pair is fixed for a checkpoint and does not change with B . Format metadata is published with the representation, and a layer containing both tiers issues one grouped GEMM per tier. A startup probe measures the largest conversion workspace; the runtime reserves it before allocating expert memory.

Arena and exchange execution. The shared expert arena stores every device-resident expert representation without layer, tier, or lease-purpose partitions. The arena may contain multiple preallocated segments. The allocator rounds requests to 256-byte alignment, selects the smallest compatible free extent, splits it, and coalesces adjacent free ranges within the same segment.

The memory manager holds named donors and binds all destination extents under one mutex. If any binding fails, it releases every new extent and donor hold before returning. It distinguishes insufficient aggregate bytes from the absence of a compatible extent. A bounded worker queue applies backpressure before reservation, and donor holds prevent concurrent exchanges from spending the same future release.

Accepted copies run on a dedicated CUDA stream. A worker writes the packed representation and any kernel-native layout into the reserved extent, then waits for the completion event before publication. Copy or publication failure rolls back a destination that never became visible.

Publication and dispatch. A published handle contains the tier, arena extent, format metadata, and a monotonically increasing version. The forward path acquires a reader reference before resolving the extent and releases it after kernel launch, recording the latest completion event for each compute stream. Publication atomically replaces the registry entry. Reclamation waits for the old handle's readers and recorded events; without a precise event, it uses a conservative device synchronization. This sequence closes the race between pointer lookup, kernel launch, and extent reuse.

The dispatcher launches groups backed by published representations before a group waiting on a transfer. Repeated uses of one destination share its copy and readiness event. The runtime logs arena occupancy, reservation failures, and exchange decisions. On NVIDIA systems, a separate monitor samples process memory through NVML every 2 ms. A run is invalid if published plus reserved arena bytes exceed B .

5 EVALUATION

We evaluate five questions. RQ1 asks whether DynaByte improves tail latency at the same expert-memory budget and quality-degradation limit. RQ2 tests whether joint online allocation is necessary. RQ3 attributes the result to counterfactual valuation, explicit donors, and the shared expert arena. RQ4 measures model accuracy and runtime cost. RQ5 identifies operating regions in which a simpler policy is sufficient.

5.1 Experimental Setup

Platform and models. The primary platform is one NVIDIA RTX A6000 with 48 GB of device memory. Table 3 reports the exact checkpoint revision, packed representation pair, measured expert footprints, and tested values of the expert-memory budget B . A checkpoint enters the study only after both representations pass the kernel-correctness tests. Non-expert parameters, KV cache, activations, conversion workspace, and a safety margin are reserved before the shared expert arena is created. Across these runs, B is the expert-memory axis. Host-to-device bandwidth is the measured copy rate on that platform, and the controlled copy-rate limit swept in Section 5.6. These two axes are the two budgets posed in Section 1.

Workloads and quality. Latency requests use length-stratified ShareGPT traces [1], including prefill-heavy, decode-heavy, and balanced-length slices. We measure the isolated model path; request queueing, tokenization, transport, and serialization are outside the timed region. Quality is measured with WikiText-2 perplexity [22] and [validated downstream tasks]. Calibration prompts determine sensitivity values, predictor width, controller constants, and baseline settings. Test prompts are disjoint, and no constant is refit on them. We report each quality task separately against the all-high reference. The reference uses FP16 for every executed expert in the same

Table 3. Models and measured representation footprints. Every value is read from a run artifact rather than inferred from nominal parameter count.

Model	Layers	Experts	Low tier	High tier	B
$\square$	$\square$	$\square$	$\square$	$\square$	$\square$
$\square$	$\square$	$\square$	$\square$	$\square$	$\square$
$\square$	$\square$	$\square$	$\square$	$\square$	$\square$

Table 4. Capabilities of the compared policies. “Online” means that the allocation may change on test requests; oracle policies are not deployable.

Policy	Fidelity	Residency	Lookahead	Online
Uniform-low	fixed	all feasible	none	no
Static-mixed	calibrated	fixed	none	no
Residency-only	fixed	adaptive	none	yes
Lookahead-only	fixed	fixed	adaptive	yes
SEQ-FR/SEQ-RF	yes	yes	yes	yes
STATIC-JOINT	yes	yes	yes	no
Fixed-partition	yes	yes	yes	yes
HOBBIT-M	miss path	adaptive	adaptive	yes
DyMoE-M	adaptive	adaptive	adaptive	yes
DynaByte	yes	yes	yes	yes
Offline oracle	yes	yes	yes	oracle

runtime and streams absent experts through the same residency path. Thus “all-high” fixes fidelity, not placement, and does not assume that every FP16 expert fits in HBM simultaneously.

Baselines. Table 4 groups policies by capability. Uniform-low, Static-mixed, Residency-only, and Lookahead-only isolate one memory use. SEQ-FR assigns fidelity before residency and lookahead; SEQ-RF reverses the order. STATIC-JOINT searches all three byte shares on calibration data and freezes the best feasible point. Fixed-partition tunes three private quotas. A deployable baseline receives the same calibration traces and search budget as DynaByte. A comparison is reportable only when all deployable policies receive the same B , representations, kernels, request stream, and transition interface. Matched implementations of HOBBIT and DyMoE are marked HOBBIT-M and DyMoE-M and are included only after their behavior is validated on the same checkpoint [13, 28]. Test-side grid search and offline exchange optimization are reported only as upper bounds.

Metrics and statistics. The primary endpoint is p95 time per output token (TPOT) subject to $\Delta Q \leq \epsilon$. We also report p99 TPOT, p95 time to first token (TTFT), SLO attainment, exposed wait, migration bytes, and eviction-induced reload bytes. Every configuration uses five warm-up iterations and 100 measured requests for each of five paired seeds. Policies receive identical request orders. Tail-latency differences use a paired bootstrap clustered by seed and request trace. We report 95% confidence intervals, the per-seed values, and the median and range across seeds. Accuracy uses Wilson intervals; paired binary outcomes use McNemar’s test, and nonbinary task metrics use a paired item bootstrap. Perplexity intervals are bootstrapped over token windows. The primary comparison is DynaByte against the strongest deployable baseline. Secondary comparisons use Holm correction. Overlapping intervals are reported as inconclusive, and scores from different quality tasks are never pooled.

[Figure placeholder]
Quality–latency frontiers by model and expert-memory budget.

Fig. 6. Quality–latency frontiers under a common expert-memory budget. Points outside $\Delta Q \leq \epsilon$ remain visible but are infeasible. Bands show paired-bootstrap 95% confidence intervals.

Table 5. End-to-end results at the fixed quality-degradation limit. Values are median across seeds with 95% paired-bootstrap intervals.

Model/B	Policy	ΔQ	p95 TPOT	p99 TPOT	p95 TTFT	SLO	Peak arena

Memory accounting and admission. Each run records published plus reserved arena bytes and process memory sampled through NVML every 2 ms. A run is invalid if arena occupancy exceeds B , if the quality-degradation limit is violated, or if its exchange log cannot reconstruct the reported allocation. Process memory is reported separately because it contains non-expert state. External fragmentation is

$$F_{\text{ext}} = 1 - \frac{\text{largest free extent}}{\text{total free arena bytes}}. \quad (16)$$

We distinguish failures caused by insufficient aggregate bytes from failures caused by the absence of a compatible extent. The artifact also records the checkpoint revision, packing recipe, controller constants, allocator alignment, random seed, and concurrent device processes.

5.2 End-to-End Quality–Latency Tradeoff

RQ1: Does joint allocation improve tail latency? For every model and B , each deployable policy sweeps only controls visible on the calibration split and uses that platform’s measured host-to-device rate. Section 5.6 varies the rate separately. Figure 6 plots the resulting nondominated quality–latency points. Table 5 reports the point selected at the same ϵ . The table includes both sequential orders and STATIC-JOINT; a controller that lets two components each spend the full B appears only as a reservation-failure diagnostic.

[State the answer to RQ1 after Figure 6 and Table 5 are populated. Report the strongest feasible baseline, absolute values, confidence intervals, and any inconclusive cases.]

5.3 Why Joint Online Allocation?

RQ2: Are joint and online decisions both necessary? Figure 7 first sweeps fidelity and lookahead byte shares at a fixed host-to-device bandwidth; residency receives the remaining bytes. It then

[Figure placeholder]

Left: feasible fidelity–lookahead heatmap. Right: allocation and latency around a workload change.

Fig. 7. Necessity of joint online allocation. The heatmap sweeps fidelity and lookahead byte shares at a fixed host-to-device bandwidth and marks quality-infeasible and memory-infeasible cells. It is not a sweep of both budgets. The time series reports the convergence interval and migration bytes accumulated after a workload change.

aligns a workload-switch trace at the change point and shows how the three byte allocations move. The comparison with SEQ-FR and SEQ-RF tests joint optimization. The comparison with calibration-side STATIC-JOINT tests online adaptation. A test-side grid oracle separates adaptation loss from search error.

[State the answer to RQ2. If the feasible optimum is not interior, a sequential baseline matches DynaByte, or Static-Joint matches it within the confidence interval, narrow the joint or online claim accordingly.]

5.4 Mechanism Attribution

RQ3: Which mechanisms account for the result? We replace one mechanism at a time while holding the workload, B , and data path fixed. Independent-value scores requests without counterfactual replay. Slack-only admission removes explicit donors. Fixed-partition replaces byte exchange with three tuned private quotas. Single-timescale merges the fast and slow updates, while No-tenure removes fidelity tenure and replacement hysteresis. A fixed-arena-partition variant isolates the allocator from the allocation policy.

[Figure placeholder]

Left: mechanism ablations. Right: accepted exchange types and realized benefit.

Fig. 8. Mechanism attribution. The ablation panel reports tail latency and migration bytes. The exchange panel reports requested and released byte uses, predicted benefit, realized benefit, and reload cost.

Table 6. Anatomy of accepted exchanges. A type that never occurs is not credited with the end-to-end result.

Request $\leftarrow$ donor	Count	Bytes	Realized gain	Reload cost
Residency $\leftarrow$ fidelity	$\square$	$\square$	$\square$	$\square$
Lookahead $\leftarrow$ fidelity	$\square$	$\square$	$\square$	$\square$
Lookahead $\leftarrow$ residency	$\square$	$\square$	$\square$	$\square$
Other accepted types	$\square$	$\square$	$\square$	$\square$

[State the answer to RQ3. Quantify the effects of counterfactual replay, explicit donors, shared allocation, and churn control separately.]

5.5 Model Accuracy and Runtime Cost

RQ4: Are the surrogates and implementation accurate enough? Figure 9 compares predicted exposed wait $\hat{L}$ with measured exposed wait. It evaluates in-overlap bandwidth against an idle-copy estimate. The quality panel compares $\hat{R}$ with measured quality degradation and reports both unsafe acceptances and conservative rejections. Predictor results include recall at envelope depth and a reliability diagram for ω_j . The runtime panels report fast- and slow-control time, allocator fragmentation, and published plus reserved bytes. The scale sweep varies candidate requests K and predicted uses W and reports the cost of the implemented replay.

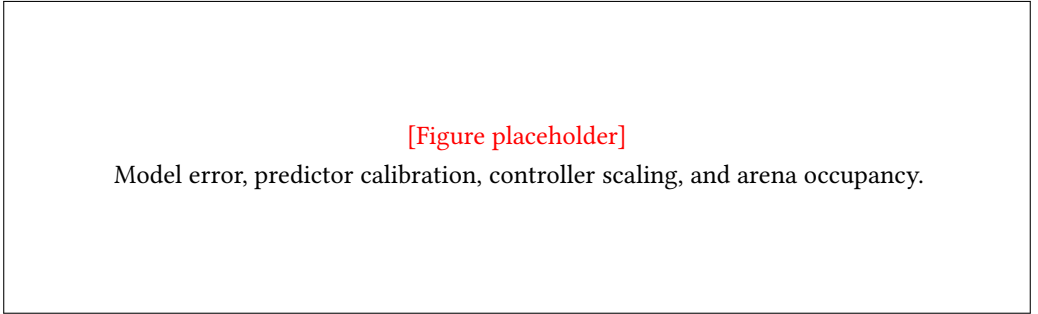


Fig. 9. Model accuracy and runtime cost. Error is separated by workload regime. The arena panel includes B , reservation failures by cause, and median, p95, and maximum F_{ext} .

[State the answer to RQ4. Report model errors in original units, misclassification counts for the quality-risk account, controller time as a fraction of layer compute, and the number of memory-invariant violations.]

5.6 Robustness and Boundary Regimes

RQ5: When is a simpler policy sufficient? We sweep B , controlled copy-rate limits, representation size, batch size, predictor noise, and ϵ . The operating-region plot marks where the selected policy reduces to uniform residency, static mixed fidelity, demand loading, or deadline-guided lookahead. It also reports regions in which DynaByte and the best static baseline are statistically indistinguishable.

[Figure placeholder]

Operating regions across budget, bandwidth, representation size, and prediction error.

Fig. 10. Robustness and boundary regimes. Hatched regions denote infeasible quality or memory configurations; neutral regions denote no statistically resolved difference from the best static policy.

[State the answer to RQ5, including no-benefit regions and negative results. Do not attribute a boundary-policy result to joint allocation.]

6 DISCUSSION

6.1 Applicability and Boundary Regimes

Joint allocation matters only when the expert-memory budget leaves a meaningful choice among fidelity, residency, and lookahead. When $\Phi \geq \rho$, every expert can remain resident at high precision and the conflict disappears. When $1 \leq \Phi < \rho$, the aggregate low-tier footprint fits, but every fidelity increment consumes bytes that could otherwise retain or stage an expert. When $\Phi < 1$, some experts must remain absent even at low precision, and a sufficiently tight budget may admit no state that satisfies the quality-degradation limit. DynaByte targets feasible operating points at which at least two memory uses retain competing value.

Host-to-device bandwidth supplies the other boundary. When the measured rate hides the transfers that residency does not already avoid, lookahead retains little additional value. When the rate is tight, a fidelity upgrade that enlarges a transfer occupies the migration queue for longer and becomes more expensive. Φ still describes only whether the low-precision or high-precision experts can all remain resident.

The footprint ratios do not determine the best allocation by themselves. Stable quality sensitivity favors persistent fidelity leases, strong reuse raises the value of residency, and predictable future use with sufficient compute overlap raises the value of lookahead. The lease model consequently reduces to simpler policies at the boundaries. It stops migration when the high-tier footprint fits. A loose quality-degradation limit and strong reuse favor low-tier residency, whereas weak reuse favors deadline-aware lookahead. Stable sensitivity and routing demand favor a static mixed-precision assignment.

The boundary regimes also delimit the contribution. If a fixed memory partition matches donor-backed exchange as the workload changes, dynamic marginal values add complexity without improving allocation. If routing-weighted sensitivity does not improve on a frozen calibration ranking, fidelity candidates need not be refreshed online. If per-expert deadline replay does not improve on a scalar-S policy, its finer urgency model is unnecessary for the tested workloads. DynaByte is intended to avoid committing to one boundary policy when the relative values change, not to outperform that policy where it is already sufficient.

6.2 Guarantees and Estimation Limits

The design separates properties enforced by the executor from quantities estimated by the controller. The quality account combines offline sensitivity q_i , which estimates the consequence of using expert i 's low representation, with routing weight w_i , which estimates how often the current workload exercises that consequence. Their product is useful for admission, but it need not be additive across experts or stable across tasks. Interactions among quantized experts can invalidate a sum of individual sensitivities, and workload shift can invalidate the calibration values. The runtime therefore treats $\hat{R}(X) \leq \hat{\epsilon}$ as a feasibility proxy and never converts risk into milliseconds. The actual constraint $\Delta Q \leq \epsilon$ is evaluated on complete configurations, with $\hat{\epsilon}$ fixed before the evaluation workload is observed.

Table 7. Enforced properties and model-dependent decisions.

Property	Status
Expert-memory budget	Enforced after the executor binds compatible arena extents for the complete transition schedule.
Publication safety	Enforced by immutable versioned handles, publish-after-copy, and delayed reclamation.
Routing semantics	Enforced: top- k choices are unchanged and an absent expert is loaded on demand.
Quality-degradation limit	Approximated by a calibrated risk account; not a per-request guarantee.
Latency value	Estimated by deadline replay; not a direct optimizer of P95 TPOT.
Online allocation	A bounded online policy; no global-optimality claim.

Latency estimates have different failure modes. A false-positive lookahead consumes transfer bandwidth and a temporary lease; a false negative becomes a demand load. Transfer-time error shifts the predicted ready time, while stale reuse or routing estimates can protect the wrong resident or fidelity lease. Observed stall, pollution, late bytes, and reloads adapt the candidate envelope and later exchange values, but feedback cannot remove the initial error. Protection also has a finite boundary: a speculative exchange must price a reload predicted inside the current envelope, whereas reuse beyond that envelope may still be missed.

Memory safety and routing semantics do not depend on prediction accuracy; latency efficiency and quality-risk admission do. If reservation or extent binding fails, the executor discards the provisional state and leaves the published state unchanged. A precision transition serves from the old representation until its versioned replacement is complete, and a missed lookahead waits on the demand path without changing the selected experts.

6.3 System Scope and Deployment Assumptions

DynaByte does not modify top- k , router parameters, or expert architecture, and it never skips a routed expert to avoid a transfer. Each checkpoint supplies one low- and one high-precision representation produced offline with a fixed quantization recipe; the runtime selects between them rather than synthesizing bit widths online. Sensitivity values and controller constants are selected on a calibration split rather than fitted to evaluation requests. Shared experts that execute unconditionally are charged to the non-expert reserve because they are not candidates for eviction or lookahead.

The hard ledger covers the shared expert arena after non-expert parameters, KV cache, activations, and workspaces receive declared reserves. A request that exceeds those reserves can exhaust process memory even while the expert ledger remains within B . The extent allocator may also refuse an

exchange when no free extent is large enough, despite sufficient aggregate slack. Reporting both ledger occupancy and process-level peak memory distinguishes these failures from a violation of the expert-memory invariant.

The current runtime manages one accelerator and uses host memory as its backing store. Host memory retains both packed representations for the process lifetime; a deployment that cannot do so would need to quantize on demand or read a representation from storage, adding a transition not modeled here. Resident-first execution also assumes that expert groups within a layer can be issued independently. A kernel with a fixed expert order reduces demand-miss overlap and requires a different latency estimate.

Expert-parallel execution and an SSD tier are outside the present design. They introduce per-device capacity, network or storage queues, and load imbalance rather than merely changing one bandwidth constant. Extending byte exchange to those settings requires adding those queues and constraints to the replay and ledger.

6.4 Validity, Reproducibility, and Ethics

The motivating measurements and trace replay in Sections 2.2–2.5 use configurations that differ from the joint runtime. They establish quality sensitivity, routing variation, and transfer pressure, not an end-to-end performance result for DynaByte. Claims based on their absolute values must be repeated under the final allocator, exchange policy, and memory accounting.

Conversational traces do not cover every serving workload. Long contexts, multimodal requests, and rapidly changing request mixtures can alter routing breadth, the KV reserve, and available overlap. Quality benchmarks sample different capabilities and may be too small to resolve modest changes, so results should retain per-task uncertainty rather than collapse all tasks into one score. Cross-device comparisons also include differences in kernels, runtime stacks, transfer paths, and quantization support. The policy consumes measured in-overlap bandwidth and kernel time; a device name alone is not treated as a causal variable.

The artifact will include the controller, shared-arena allocator, lease and exchange traces, check-point revisions, quantization recipes, calibration splits, random seeds, and one configuration file per reported point. Each run records published and reserved bytes, bound arena extents, refused reservations, selected expert states, and process-level peak memory. A result is reproducible under Eq. (2) only when these records suffice to verify B , ϵ , and the executed state transitions.

The study uses public latency traces and quality benchmarks, recruits no participants, collects no new user data, and sends no prompts to a third-party service. Released artifacts will identify source prompts without redistributing material whose license forbids it.

7 RELATED WORK

Prior systems control expert precision, residency, or transfer timing, often while holding the other decisions fixed. We group them by that control decision, then compare DynaByte directly with systems that already combine the mechanisms.

Precision assignment. GPTQ, AWQ, SmoothQuant, and LLM.int8() fix representations before serving [7, 11, 21, 31]. MoE-specific methods use expert sensitivity, activation frequency, or router statistics to assign precision under a memory or quality target [3, 4, 8, 16, 35]. Such methods improve fidelity for a chosen footprint but commonly evaluate the assignment under fixed placement and transfer policies. DynaByte can use their packed representations and offline sensitivities; its additional question is whether a fidelity increment is worth the residency or lookahead displaced at runtime. Mixed-precision kernels such as MxMoE remain complementary [8]: they determine the compute cost of a selected representation, not which state should occupy expert memory.

Residency and placement. Expert caches retain recently or frequently used experts and select victims using routing history, layer position, or co-activation [2, 17, 20, 32, 34, 37]. They primarily estimate residency value with a fixed expert representation. Placement methods instead colocate co-activated experts or map them according to topology and load [5, 33]; their multi-device communication objective is outside our single-accelerator implementation. In DynaByte, a residency lease is valued together with the fidelity or lookahead request that would displace it, so eviction cost belongs to the complete exchange rather than to an independent cache policy.

Prefetch and mixed-precision offloading. Cross-layer prefetchers predict future experts and overlap transfers with current computation, using either fixed lookahead or adaptation from observed demand and system state [9, 14, 27]. ExpertFlow coordinates expert scheduling and movement under offload [25]; a separate work with the same name changes token assignment to improve activation efficiency [12]. Their principal control variable is transfer timing for a given expert size and quality policy.

HOBBIT combines dynamic mixed-precision loading, adaptive prefetching, and a multi-level cache; its low-precision miss path reduces transfer cost for experts judged less critical [28]. Dy-MoE combines online expert importance, depth-aware mixed precision, caching, and lookahead prefetching, including configurations that retain low-bit experts or omit selected experts [13]. Their designs establish that mixed precision, caching, and prefetching can coexist, so component composition is not a novelty claim for DynaByte. The distinction is whether one exchange, under one quality-degradation limit, decides both which bytes occupy expert memory and which transfers occupy the migration queue. DynaByte preserves top- k semantics and represents all three uses as byte leases. Each request names its donor leases, and one deadline replay compares the resulting states under the quality-risk and byte accounts. Matched-policy evaluation must establish this distinction; terminology or device class cannot.

Serving and memory management. vLLM and SGLang manage request scheduling and KV-cache memory, while FlexGen schedules dense-model computation and transfers across a memory hierarchy [18, 26, 36]. DynaByte operates on the expert-memory budget left after non-expert state is reserved and is complementary to those serving functions. Because a serving engine can outgrow its declared KV or activation reserve, we report the runtime ledger and process-level peak separately.

8 CONCLUSION

Under a quality-degradation limit, DynaByte allocates expert memory and host-to-device bandwidth among fidelity, residency, and lookahead to reduce MoE inference tail latency. Its donor-backed byte exchanges expose the opportunity cost of each request, while deadline replay prevents residency and lookahead from receiving credit for the same avoided transfer. One admission path refreshes residency and lookahead faster than fidelity, through a shared byte ledger and a publish-after-copy protocol. The resulting design preserves routing semantics, enforces peak expert-memory use during transitions, and reduces to simpler policies when one purpose dominates.

REFERENCES

- [1] Anonymous Contributors. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Hugging Face dataset.
- [2] Shiyi Cao et al. 2024. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. *arXiv preprint arXiv:2411.11217* (2024).
- [3] Krishna Teja Chitty-Venkata, Jie Ye, and Murali Emani. 2025. MoPEQ: Mixture of Mixed Precision Quantized Experts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4023–4032.

- [4] Mohammed Nowaz Rabbani Chowdhury, Kaoutar El Maghraoui, Hsinyu Tsai, Naigang Wang, Geoffrey W. Burr, Liu Liu, and Meng Wang. 2026. Efficient Quantization of Mixture-of-Experts with Theoretical Generalization Guarantees. In *The Fourteenth International Conference on Learning Representations*.
- [5] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [6] DeepSeek-AI. 2024. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [7] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, Vol. 35. 30318–30332.
- [8] Haojie Duanmu, Xuhong Li, Zhihang Yuan, Size Zheng, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2025. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. *arXiv preprint arXiv:2505.05799*. [arXiv:2505.05799](#) [cs.LG]
- [9] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [11] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [12] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2024. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. *arXiv preprint arXiv:2410.17954* (2024).
- [13] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. 2026. DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge. *arXiv preprint arXiv:2603.19172*. [arXiv:2603.19172](#) [cs.LG]
- [14] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
- [15] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chait, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [16] Young Jin Kim, Raffy Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv preprint arXiv:2310.02410*. [arXiv:2310.02410](#) [cs.LG]
- [17] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6710–6720.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [19] Dmitry Lepikhin, Hyukjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.
- [20] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive gating in mixture-of-experts based language models. *arXiv preprint arXiv:2310.07188* (2023).
- [21] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of machine learning and systems* 6 (2024), 87–100.
- [22] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. [arXiv:1609.07843](#) [cs.CL]
- [23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [24] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.
- [25] Zixu Shen, Kexin Chu, Yifan Zhang, Dawei Xiang, Runxin Wu, and Wei Zhang. 2025. ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference. *arXiv preprint arXiv:2510.26730*.

- arXiv:2510.26730 [cs.PF]
- [26] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. *Proceedings of Machine Learning and Systems* 5 (2023), 1–16.
 - [27] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134* (2024).
 - [28] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433* (2024).
 - [29] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL]
 - [30] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
 - [31] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*. PMLR, 38087–38099.
 - [32] Lelang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361* (2024).
 - [33] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K DK Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 915–925.
 - [34] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. EdgeMoE: Fast on-device inference of MoE-based large language models. *arXiv preprint arXiv:2308.14352* (2023).
 - [35] Jinhao Zhang, Yunquan Zhang, Boyang Zhang, Zeyu Liu, and Daning Cheng. 2025. MoQE: Improve Quantization Model performance via Mixture of Quantization Experts. *arXiv preprint arXiv:2508.09204* (2025).
 - [36] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Liang Shi, Chenguang Wu, et al. 2024. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104* (2024).
 - [37] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.